

4.2 Snake game

Snake Game

Overview

This tutorial builds a classic Snake game using ESP32, OLED display (SSD1306), and joystick module. The game features snake movement controlled by joystick, food generation, score tracking, and game over detection.

Component Required

(1) x Elegoo ESP32

(1) x 128×64 OLED Display (SSD1306)

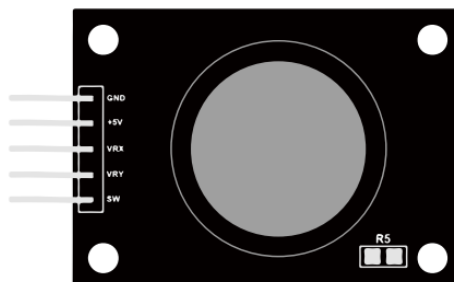
(1) x Joystick Module

Component Introduction

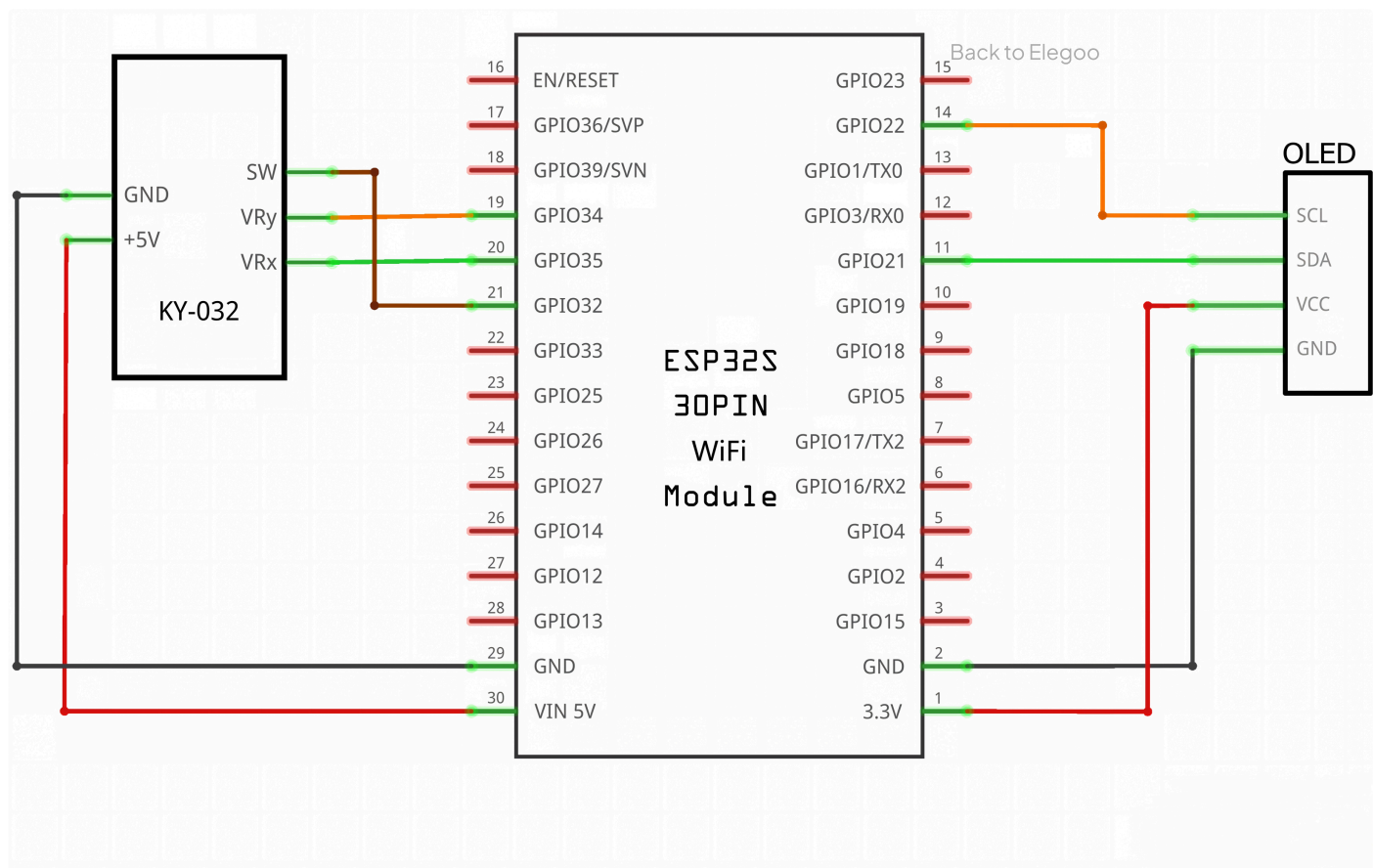
Joystick

The module has 5 pins: VCC, Ground, X, Y, Key. Note that the labels on yours may be slightly different, depending on where you got the module from. The thumb stick is analog and should provide more accurate readings than simple 'directional' joysticks that use some forms of buttons, or mechanical switches. Additionally, you can press the joystick down (rather hard on mine) to activate a 'press to select' push-button.

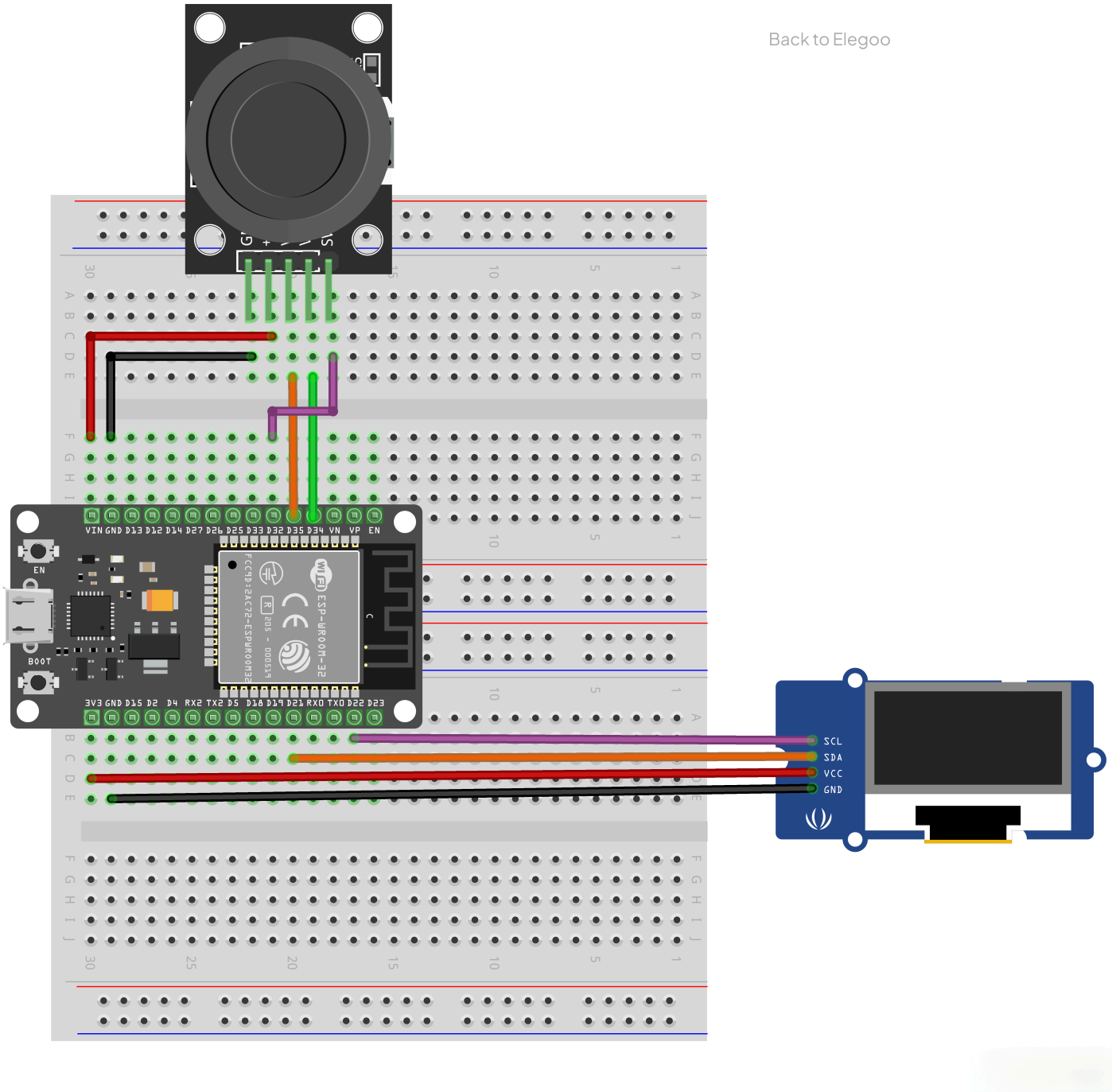
We have to use analog Arduino pins to read the data from the X/Y pins, and a digital pin to read the button. The Key pin is connected to ground, when the joystick is pressed down, and is floating otherwise. To get stable readings from the Key /Select pin, it needs to be connected to VCC via a pull-up resistor. The built in resistors on the Arduino digital pins can be used. For a tutorial on how to activate the pull-up resistors for Arduino pins, configured as inputs. For example: `pinMode(SW_pin, INPUT);`



Connection Schematic



Wiring diagram



Connection Schematic

Component	ESP32 Pin	Component	ESP32 Pin
OLED VCC	3.3V	OLED GND	GND
OLED SDA	GPIO 21	OLED SCL	GPIO 22
Joystick VCC	3.3V	Joystick GND	GND
Joystick X	GPIO 35	Joystick Y	GPIO 34
Joystick Button	GPIO 32		

Program Architecture

```
1 snake_game.ino (Main Program)
2   |—— Calls → OLED Module (Adafruit_SSD1306)
3   |—— Calls → Snake Module
4   |   |—— snake.h (Macro Declarations & Global Variables)
5   |   |—— snake.cpp (Snake Movement & Drawing)
6
```

7		— Calls → Joystick Module
8		— Joystick.h (Pin Definitions)
9		— Joystick.cpp (Joystick Input Handling)
10		— Calls → Food Module
11		— food.cpp (Food Generation & Collision)
12		— Calls → Score Module
13		— score.h (Score Declarations)
		— score.cpp (Score Management)

[Back to Elegoo](#)

Code Explanation

 [DOWNLOAD snake_game.zip](#)

1. Header File: snake.h

Function: Defines hardware configuration, game parameters and global variables, serving as the core interface file for the entire project.

Hardware Configuration:

```
1 | #define OLED_WIDTH 128
2 | #define OLED_HEIGHT 64
3 | #define OLED_ADDR 0x3C
```

- ▶ [OLED_WIDTH/OLED_HEIGHT](#) : OLED display width and height (128×64 pixels)
- ▶ [OLED_ADDR](#) : I2C address of OLED, default 0x3C

Game Parameters:

```
1 | #define DEAD_ZONE 600
2 | #define MOVE_STEP 1
3 | #define MOVE_SPEED 50
4 | #define SNAKE_SIZE 4
5 | #define MAX_LENGTH 30
```

- ▶ [DEAD_ZONE](#) : Joystick dead zone threshold to prevent jitter when centered
- ▶ [MOVE_STEP](#) : Snake movement step size (1 pixel)
- ▶ [MOVE_SPEED](#) : Game speed (50ms), smaller value means faster
- ▶ [SNAKE_SIZE](#) : Size of snake segments and food (4×4 pixels)
- ▶ [MAX_LENGTH](#) : Maximum snake length limit

Global Variables:

```
1 | extern int snake_x[MAX_LENGTH];
2 | extern int snake_y[MAX_LENGTH];
3 | extern int snake_length;
4 | extern int current_dir;
```

- ▶ [snake_x/snake_y](#) : Arrays storing coordinates of each snake segment (index 0 is head)
- ▶ [snake_length](#) : Current snake length
- ▶ [current_dir](#) : Current movement direction (-1=none, 0=up, 1=right, 2=down, 3=left)

2. Implementation File: snake.cpp

(1) resetSnake()

Function: Resets snake to initial state, including position, length, direction, etc.

```
1 | snake_x[0] = (OLED_WIDTH - SNAKE_SIZE) / 2;
2 | snake_y[0] = (OLED_HEIGHT - SNAKE_SIZE) / 2;
```

- ▶ Moves snake head to screen center (X: (128-4)/2=62, Y: (64-4)/2=30)

```
1 | snake_length = 1;
```

[Back to Elegoo](#)

- Resets snake length to 1 (only head remains)

```
1 | current_dir = -1;
```

- Clears current movement direction, making snake stationary

```
1 | isShow = true;
```

- Resets snake head blink state to visible

```
1 | if(!start_game){ isGameOver=true; }
```

- If game has already started (not first launch), sets game over flag

(2) moveSnake()

Function: Controls snake movement, including body following and head movement.

```
1 | if (current_dir == -1) return;
```

- Returns directly without movement when no direction input

```
1 | for (int i = snake_length - 1; i > 0; i--) {  
2 |     snake_x[i] = snake_x[i - 1];  
3 |     snake_y[i] = snake_y[i - 1];  
4 | }
```

- Snake body following algorithm: traverses from tail to head, each segment inherits previous segment's coordinates

```
1 | switch (current_dir) {  
2 |     case 0: snake_y[0] -= MOVE_STEP + current_score/5; break;  
3 |     case 1: snake_x[0] += MOVE_STEP + current_score/5; break;  
4 |     case 2: snake_y[0] += MOVE_STEP + current_score/5; break;  
5 |     case 3: snake_x[0] -= MOVE_STEP + current_score/5; break;  
6 | }
```

- Moves snake head based on direction: 0=up (Y decreases), 1=right (X increases), 2=down (Y increases), 3=left (X decreases)
- Movement speed increases with score: `MOVE_STEP + current_score/5`, implementing difficulty progression

(3) checkSnakeOverBoundary()

Function: Detects if snake head crosses boundary, returns true to trigger game reset if out of bounds.

```
1 | bool xOver = (snake_x[0] < 0) || (snake_x[0] > MAX_SNAKE_X);
```

- X-axis boundary check: snake head X coordinate less than 0 or greater than max boundary (124)

```
1 | bool yOver = (snake_y[0] < 8) || (snake_y[0] > MAX_SNAKE_Y+2);
```

- Y-axis boundary check: snake head Y coordinate less than 8 (reserved for score display) or greater than max boundary (62)

(4) drawSnake()

Function: Draws snake body and blinking snake head.

```
1 | for (int i = 1; i < snake_length; i++) {  
2 |     display.fillRect(snake_x[i], snake_y[i], SNAKE_SIZE, SNAKE_SIZE, SNAKE_BODY_COLOR);  
3 | }
```

- Draws snake body: starts from index 1 (skipping head), each segment is a 4×4 white rectangle

```

1 | if (isShow){
2 |     display.fillRect(snake_x[0], snake_y[0], SNAKE_SIZE, SNAKE_SIZE, SNAKE_HEAD_COLOR);
3 | }else{
4 |     display.fillRect(snake_x[0], snake_y[0], SNAKE_SIZE, SNAKE_SIZE, SSD1306_BLACK);
5 | }
```

- Draws snake head: determines to show white (visible) or black (hidden) based on `isShow` flag, creating blinking effect

(5) `blinkSnakeAndFood()`

Function: Completes one frame drawing including score, snake, food, and toggles blink state.

```

1 | display.clearDisplay();
```

- Clears display buffer

```

1 | drawScore();
```

- Draws score at top of screen

```

1 | drawSnake();
```

- Draws snake (including blinking head)

```

1 | drawFood();
```

- Draws food

```

1 | display.display();
```

- Refreshes buffer content to OLED screen

```

1 | isShow = !isShow;
```

- Toggles snake head blink state, takes effect next frame

3. Header File: `Joystick.h`

Function: Defines joystick pin macros and button detection function declarations.

```

1 | #define JOY_X_PIN 35
2 | #define JOY_Y_PIN 34
3 | #define JOY_BUTTON_PIN 32
```

- `JOY_X_PIN` : Joystick X-axis analog input pin (GPIO 35)
- `JOY_Y_PIN` : Joystick Y-axis analog input pin (GPIO 34)
- `JOY_BUTTON_PIN` : Joystick button digital input pin (GPIO 32)

4. Implementation File: `Joystick.cpp`

(1) `initJoystick()`

Function: Initializes joystick pin input modes.

```
1 | pinMode(JOY_X_PIN, INPUT);
2 | pinMode(JOY_Y_PIN, INPUT);
```

[Back to Elegoo](#)

- Sets X/Y axis pins as analog input mode

```
1 | pinMode(JOY_BUTTON_PIN, INPUT_PULLUP);
```

- Sets button pin as digital input with internal pull-up

(2) readJoystick()

Function: Reads joystick analog values, determines movement direction and updates `current_dir`.

```
1 | int xVal = analogRead(JOY_X_PIN);
2 | int yVal = analogRead(JOY_Y_PIN);
```

- Reads X/Y axis analog values (ESP32 12-bit ADC, range 0–4095)

```
1 | const int MAX_VAL = 4095;
```

- ADC maximum value constant

```
1 | if (yVal < DEAD_ZONE && current_dir != 2) { current_dir = 0; }
```

- Y value less than dead zone and current direction is not down (2), set to up (0)

```
1 | else if (xVal > MAX_VAL - DEAD_ZONE && current_dir != 3) { current_dir = 1; }
```

- X value greater than max minus dead zone and current direction is not left (3), set to right (1)

```
1 | else if (yVal > MAX_VAL - DEAD_ZONE && current_dir != 0) { current_dir = 2; }
```

- Y value greater than max minus dead zone and current direction is not up (0), set to down (2)

```
1 | else if (xVal < DEAD_ZONE && current_dir != 1) { current_dir = 3; }
```

- X value less than dead zone and current direction is not right (1), set to left (3)

(3) isJoystickButtonPressed()

Function: Detects if joystick button is pressed.

```
1 | return digitalRead(JOY_BUTTON_PIN) == LOW;
```

- Reads button pin state, returns true if LOW (button pressed and grounded)

5. Implementation File: food.cpp

(1) generateFood()

Function: Generates food coordinates within valid range that don't overlap with snake body.

```
1 | food_x = (random(0, MAX_SNAKE_X / SNAKE_SIZE - 2)) * SNAKE_SIZE;
2 | food_y = (random(4, MAX_SNAKE_Y / SNAKE_SIZE)) * SNAKE_SIZE;
```

- Generates random coordinates, aligned to SNAKE_SIZE (4) multiple for grid alignment

```

1 | for (int i = 0; i < snake_length; i++) {
2 |   if (abs(snake_x[i] - food_x) < SNAKE_SIZE && abs(snake_y[i] - food_y) < SNAKE_SIZE) {
3 |     overlap = true; break;
4 |   }
5 | }

```

- Checks if food overlaps with any snake segment, regenerates if overlapping

(2) drawFood()

Function: Draws circular food at food coordinates.

```

1 | int center_x = food_x + FOOD_SIZE / 2;
2 | int center_y = food_y + FOOD_SIZE / 2;

```

- Calculates food center point coordinates

```

1 | display.fillCircle(center_x, center_y, FOOD_SIZE / 2, FOOD_COLOR);

```

- Draws filled circle with center point and radius of 2

(3) checkFoodCollision()

Function: Detects if snake head eats food, snake grows and returns true if eaten.

```

1 | bool xOverlap = (snake_x[0] < food_x + FOOD_SIZE/2) && (snake_x[0] + SNAKE_SIZE > food_x + FOOD_SIZE/2);
2 | bool yOverlap = (snake_y[0] < food_y + FOOD_SIZE/2) && (snake_y[0] + SNAKE_SIZE > food_y + FOOD_SIZE/2);

```

- Collision detection: checks if snake head overlaps with food center point

```

1 | if (snake_length < MAX_LENGTH) {
2 |   int tail_idx = snake_length - 1;
3 |   int new_tail_x = snake_x[tail_idx];
4 |   int new_tail_y = snake_y[tail_idx];

```

- Gets current tail coordinates as base position for new segment

```

1 | switch (current_dir) {
2 |   case 0: new_tail_y += SNAKE_SIZE; break;
3 |   case 1: new_tail_x -= SNAKE_SIZE; break;
4 |   case 2: new_tail_y -= SNAKE_SIZE; break;
5 |   case 3: new_tail_x += SNAKE_SIZE; break;
6 | }

```

- Extends new segment in opposite direction of movement from tail (moving up → new segment below)

```

1 | snake_x[snake_length] = new_tail_x;
2 | snake_y[snake_length] = new_tail_y;
3 | snake_length++;

```

- Stores new segment coordinates in array, increments snake length

6. Header File: score.h

Function: Declares score variable and related function interfaces.

```

1 | extern int current_score;
2 | void initScore();
3 | void addScore(int val);
4 | void resetScore();
5 | void drawScore();

```

7. Implementation File: score.cpp

[Back to Elegoo](#)

(1) initScore()

Function: Initializes score to 0 (called at game start).

```
1 | current_score = 0;
```

(2) addScore(int val)

Function: Adds specified score value (called when food is eaten).

```
1 | current_score += val;
```

- Adds parameter `val` to current score

(3) resetScore()

Function: Resets score to 0 (called at game over).

```
1 | current_score = 0;
```

(4) drawScore()

Function: Draws current score at top-left corner of OLED.

```
1 | display.setTextSize(1);
```

- Sets font size to 1 (8×8 pixels)

```
1 | display.setTextColor(SSD1306_WHITE);
```

- Sets text color to white

```
1 | display.setCursor(2, 2);
```

- Sets cursor position to (2, 2), 2 pixels margin from top-left

```
1 | display.print("Score: ");  
2 | display.print(current_score);
```

- Outputs "Score: " prefix and current score value

8. Main File: snake_game.ino

Global Objects:

```
1 | Adafruit_SSD1306 display(OLED_WIDTH, OLED_HEIGHT, &Wire, -1);
```

- Creates OLED display object with specified dimensions, I2C bus and reset pin (-1 means not used)

setup()

Function: Initializes system hardware and game state.

```
1 | Serial.begin(115200);
```

- Initializes serial communication at 115200 baud rate

```
1 | display.begin(SSD1306_SWITCHCAPVCC, OLED_ADDR);
```

[Back to Elegoo](#)

- Initializes OLED display with internal charge pump power supply

```
1 | initJoystick();
```

- Initializes joystick module

```
1 | initScore();
```

- Initializes score

```
1 | resetSnake();
```

- Initializes snake state

```
1 | generateFood();
```

- Generates initial food

loop()

Function: Main game loop, handles game states and logic.

Start Screen:

```
1 | if(start_game){
2 |   display.print("press joy button");
3 |   if (isJoystickButtonPressed()) { start_game=false; }
4 |   return;
5 | }
```

- Shows "press joy button" on first launch, waits for button press to start game

Game Over Screen:

```
1 | if (isGameOver){
2 |   display.print("GAME");
3 |   display.print("OVER");
4 |   if (isJoystickButtonPressed()){
5 |     resetSnake(); generateFood(); resetScore();
6 |     isGameOver = false;
7 |   }
8 |   return;
9 | }
```

- Shows "GAME OVER" when game ends, waits for button press to restart

Main Game Logic:

```
1 | readJoystick();
```

- Reads joystick input, updates movement direction

```
1 | moveSnake();
```

- Moves snake according to direction

```
1 | if (checkSnakeOverBoundary()) { resetSnake(); generateFood(); resetScore(); }
```

[Back to Elegoo](#)

- Detects boundary collision, resets game if out of bounds

```
1 | if (checkFoodCollision()) { generateFood(); addScore(1); }
```

- Detects food collision, generates new food and adds score if eaten

```
1 | blinkSnakeAndFood();
```

- Draws current frame

```
1 | delay(MOVE_SPEED);
```

- Delay controls game speed

Code Modification Tips

Modify Movement Range (Boundary Detection)

Location: snake.h

```
1 | #define MAX_SNAKE_X (OLED_WIDTH - SNAKE_SIZE)
2 | #define MAX_SNAKE_Y (OLED_HEIGHT - SNAKE_SIZE)
```

Function: `checkSnakeOverBoundary()`

```
1 | bool yOver = (snake_y[0] < 8) || (snake_y[0] > MAX_SNAKE_Y+2);
```

- `y < 8` : Top boundary, reserved for score display area
- Modify this value to adjust the upper/lower game area boundaries

Modify Score Rules

Location: `addScore()` in score.cpp

```
1 | void addScore(int val) {
2 |     current_score += val;
3 | }
```

Call Location: snake_game.ino

```
1 | if (checkFoodCollision()) { generateFood(); addScore(1); }
```

- Change `addScore(1)` to `addScore(10)` for 10 points per food
- Add `current_score += val * multiplier` in `addScore()` for multiplier scoring

Modify Game Speed

Base Speed Parameters: snake.h

```
1 | #define MOVE_SPEED 50 // Unit: milliseconds, smaller = faster
2 | #define MOVE_STEP 1 // Pixels per frame
```

Dynamic Speed: `moveSnake()` in snake.cpp

```
1 | case 0: snake_y[0] -= MOVE_STEP+current_score/5; break;
```

[Back to Elegoo](#)

- `current_score/5` : Speed increase coefficient based on score
- Change to `current_score/10` for slower speed progression
- Change to `current_score/2` for faster speed progression

Frame Rate Control: snake_game.ino

```
1 | delay(MOVE_SPEED);
```

- Modify `MOVE_SPEED` to directly adjust game frame rate

Modify Snake and Food Size

Location: snake.h

```
1 | #define SNAKE_SIZE 4
2 | #define FOOD_SIZE 4
```

- Larger size makes snake and food more visible but reduces play area
- Smaller size increases difficulty and available space

Modify Maximum Snake Length

Location: snake.h

```
1 | #define MAX_LENGTH 30
```

- Modify this value to limit maximum snake length
- Increase to extend game duration

Modify Joystick Dead Zone

Location: snake.h

```
1 | #define DEAD_ZONE 600
```

- ESP32 ADC range is 0–4095
- Increase `DEAD_ZONE` to reduce jitter when joystick is centered
- Decrease `DEAD_ZONE` to increase joystick sensitivity

Modify Joystick Button Function

Location: `isJoystickButtonPressed()` in joystick.cpp

```
1 | bool isJoystickButtonPressed() {
2 |     return digitalRead(JOY_BUTTON_PIN) == LOW;
3 | }
```

Call Location: snake_game.ino

```
1 | if (isJoystickButtonPressed()) { start_game=false; }
```

- Current button function: start/restart game
- Can add pause function: detect button in main loop and toggle pause state

Modify Display Style

Score Display: score.cpp

```
1 | display.setTextSize(1); // Modify font size
2 | display.setCursor(2, 2); // Modify display position
3 | display.print("Score: "); // Modify display text
```

[Back to Elegoo](#)

Snake Head Blinking: snake.cpp

```
1 | isShow = !isShow; // Controls blinking in blinkSnakeAndFood()
```

- Remove this line to disable snake head blinking

3D model

The downloaded ZIP file contains 3D printing files, which you can use to print the housing by yourself as needed.

[← meteorological station](#)

[smart access control→](#)